INFORME DE PROYECTO

Sistema IoT con microcontrolador, sensores ambientales y comunicación MQTT

Integrantes	Kevin F. González Orduz Neiver Y. Granados Quintero
Asignatura	IoT (Internet de las Cosas)
Docente	Eliecer Montero
Programa	Tecnología en Desarrollo de Sistemas Informáticos
Año	2026

Documento organizado según los criterios de la rúbrica de evaluación de la Semana 10: protocolos de comunicación para IoT (HTTP y MQTT).

1. Introducción

El Internet de las Cosas (IoT) permite que dispositivos físicos obtengan información mediante sensores y la compartan por una red para facilitar su supervisión. En esta práctica se utiliza una placa de desarrollo NodeMCU basada en ESP8266, sensores ambientales, indicadores LED y comunicación Wi-Fi con el protocolo MQTT. La información se observa desde el monitor serial del Arduino IDE y desde MQTT Explorer, conectado al broker público broker.hivemq.com.

Precisión sobre el hardware: en la fotografía se identifica una placa NodeMCU 1.0 (ESP-12E/ESP8266), no una ESP32. Por ello, este informe describe el montaje observado como NodeMCU/ESP8266. Si el grupo utilizó otra placa en una etapa diferente, debe confirmarse antes de entregar el documento.

La evidencia aportada muestra publicaciones de temperatura, humedad y luminosidad, además de mensajes recibidos de otros equipos. Los apartados que no se pueden confirmar directamente con las capturas —por ejemplo, una base de datos histórica o un dashboard móvil— se presentan como propuestas de ampliación y no como funciones ya implementadas.

2. Objetivos

Objetivo general. Documentar un prototipo IoT que obtiene datos ambientales, los transmite mediante MQTT y permite observarlos en tiempo real.

Objetivos específicos. Identificar los componentes del circuito; describir la conexión al broker y los tópicos utilizados; evidenciar la publicación y recepción de mensajes; comparar MQTT con HTTP; y proponer una estructura de tópicos, niveles QoS y flujo de datos para ampliar el sistema.

3. Componentes del montaje

En la imagen se observan una placa NodeMCU ESP8266, una protoboard, cables jumper, indicadores LED con resistencias y módulos sensores. El módulo azul con rejilla corresponde visualmente a un sensor de la familia DHT, aunque la referencia exacta debe confirmarse físicamente. También se aprecia un módulo fotosensible, pero el modelo y el tipo de salida deben verificarse en el componente.

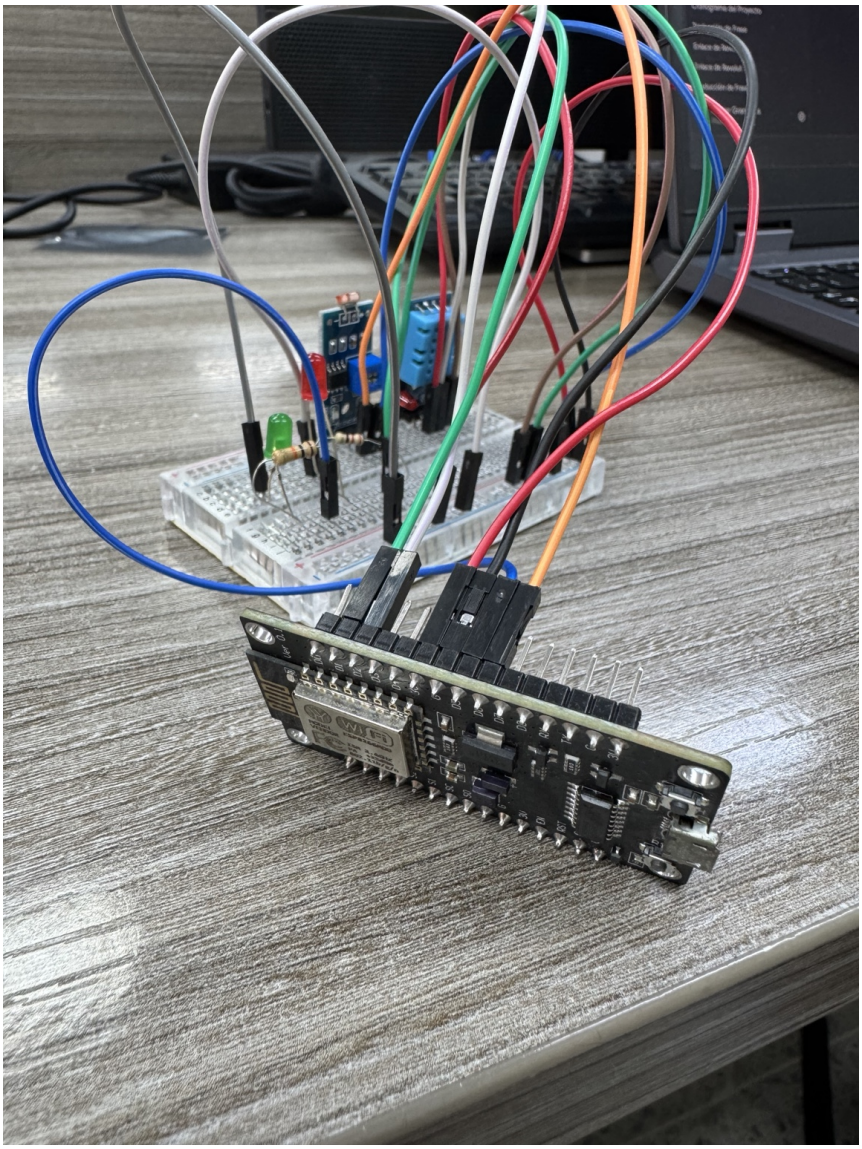


Figura 1. Montaje físico del prototipo IoT: placa NodeMCU, protoboard, sensores, LED y cableado.

Recomendaciones eléctricas: verificar que los módulos compartan GND, que la alimentación sea compatible con cada componente y que los pines del circuito coincidan con los definidos en el programa. Las fotografías no permiten establecer con certeza todas las conexiones pin a pin.

4. Funcionamiento y comunicación MQTT

El sistema sigue esta secuencia general: (1) los sensores entregan mediciones; (2) el microcontrolador procesa las lecturas; (3) se conecta a la red Wi-Fi; (4) publica los valores en tópicos MQTT; y (5) MQTT Explorer muestra los mensajes que llegan al broker. El monitor serial permite revisar los valores enviados y los mensajes recibidos de otros equipos.

4.1. Conexión al broker

La captura de MQTT Explorer muestra la conexión a **broker.hivemq.com**. Para una conexión MQTT TCP sin cifrado, el puerto habitual es 1883; debe coincidir con la configuración del programa y del cliente. La evidencia visual de MQTT Explorer y los registros del monitor serial respaldan que se estaban intercambiando mensajes durante la prueba.

Al ser un broker público compartido, los tópicos deben tener un prefijo identificador del equipo y no se deben publicar contraseñas, datos personales ni información sensible.

4.2. Evidencia del monitor serial y MQTT Explorer

En el monitor serial se observan lecturas de temperatura cercanas a 29,9 °C, humedad de 59 % y luminosidad con valor 0. También aparecen mensajes recibidos en tópicos de otros grupos, por ejemplo, bajo las rutas de temperatura de granados_gonzalez, gabriel_gomez y kevinDiana. En MQTT Explorer aparecen los tópicos del equipo y valores como estado = online, temperatura = 29.4, humedad = 59.0 y luminosidad = 0.

Las pequeñas diferencias entre la temperatura del monitor serial y la mostrada en MQTT Explorer pueden deberse a que las capturas corresponden a instantes distintos o a la frecuencia de publicación. No deben interpretarse como una medición simultánea exacta.

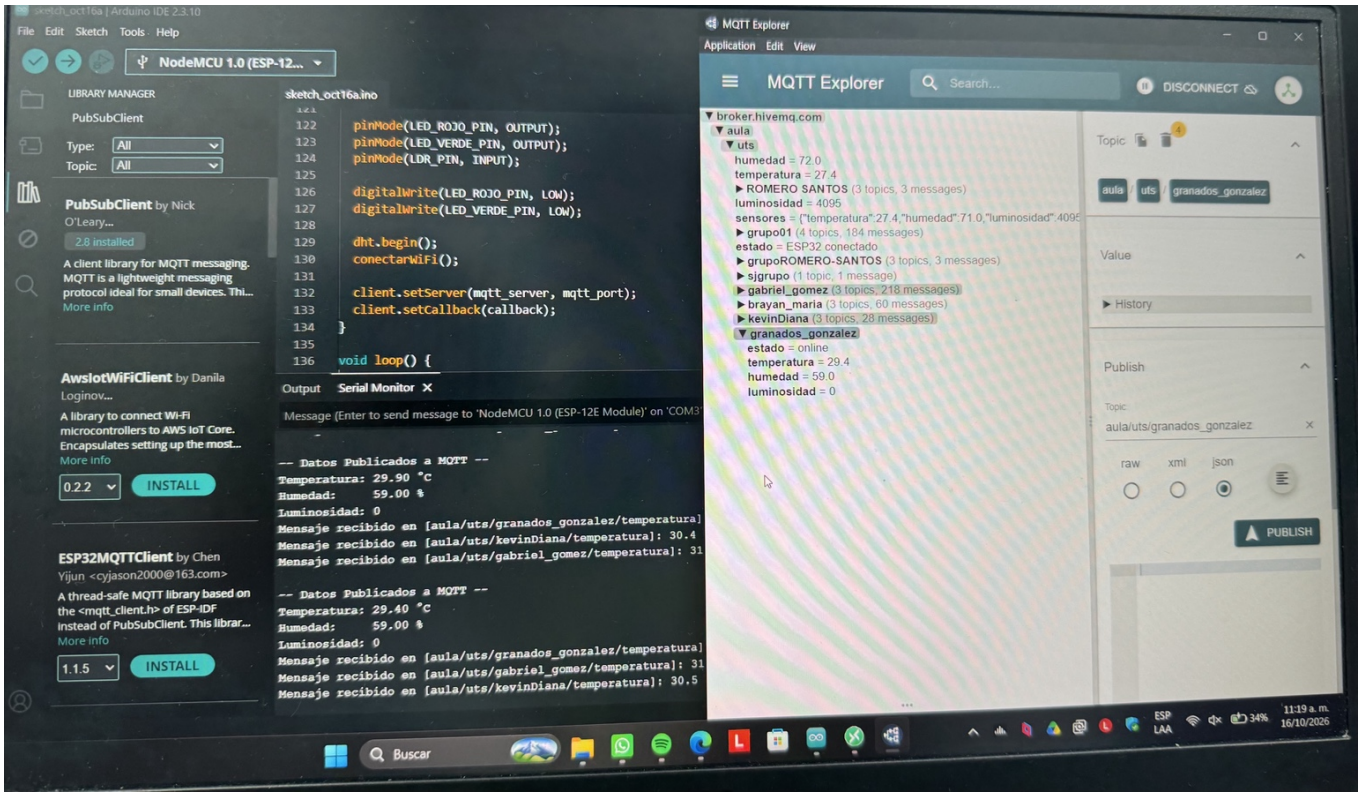


Figura 2. Evidencia de ejecución: Arduino IDE con monitor serial y MQTT Explorer mostrando tópicos y mensajes.

Resultado demostrado: se observan datos de sensores publicados y mensajes recibidos de otros equipos. La captura no demuestra por sí sola el almacenamiento en base de datos ni una aplicación móvil; esos elementos

requieren configuración adicional.

5. Tópicos MQTT

Los tópicos organizan los mensajes mediante rutas jerárquicas. En la evidencia se distingue el prefijo **aula/uts/granados_gonzalez** y se muestran los campos temperatura, humedad, luminosidad y estado. Para mantener una estructura coherente, se recomienda documentarlos así:

Tópico	Contenido	Estado
aula/uts/granados_gonzalez/temperatura	Temperatura en °C	Visible en la evidencia
aula/uts/granados_gonzalez/humedad	Humedad relativa (%)	Visible en la evidencia
aula/uts/granados_gonzalez/luminosidad	Lectura del sensor de luz	Visible en la evidencia
aula/uts/granados_gonzalez/estado	Estado general, por ejemplo online	Visible en la evidencia
aula/uts/granados_gonzalez/actuadores/led_rojo	Estado o comando para LED rojo	Propuesto; validar publicación
aula/uts/granados_gonzalez/actuadores/buzzer	Comando para buzzer, si está instalado	Propuesto; no confirmado en la foto

La rúbrica solicita publicar al menos tres tópicos personalizados. Las capturas respaldan los tópicos de temperatura, humedad y luminosidad del equipo; estado también aparece en MQTT Explorer. Los tópicos de actuadores se incluyen como diseño recomendado y solo deben marcarse como implementados si el código y una captura lo confirman.

6. Suscripción a otros equipos

El monitor serial muestra mensajes recibidos desde tópicos de otros equipos, entre ellos rutas asociadas a granados_gonzalez, gabriel_gomez y kevinDiana. Esto constituye evidencia de recepción de mensajes publicados por otros clientes. Para una documentación reproducible, se puede suscribir el cliente al filtro **aula/uts/#** o a dos tópicos concretos, siempre que los demás equipos publiquen bajo ese prefijo.

7. Monitoreo en tiempo real

MQTT Explorer permite observar la jerarquía de tópicos y los valores más recientes, mientras que el monitor serial muestra los mensajes recibidos y las lecturas publicadas por el microcontrolador. Para fortalecer la evidencia de tiempo real, se recomienda tomar una captura adicional tras varios ciclos de publicación, mostrando cómo cambian los valores y conservando la fecha o el contexto de la prueba.

8. MQTT frente a HTTP

MQTT funciona con un modelo publicador/suscriptor: el dispositivo publica un mensaje en un tópico y el broker lo distribuye a los clientes suscritos. Su cabecera pequeña y la posibilidad de mantener conexiones abiertas lo hacen conveniente para enviar mediciones frecuentes desde dispositivos con recursos limitados. HTTP, en cambio, suele utilizar un modelo de solicitud/respuesta y resulta especialmente útil para APIs y servicios web.

Para este prototipo, MQTT es apropiado porque permite enviar temperatura, humedad y luminosidad periódicamente y recibir datos de varios equipos mediante suscripciones. HTTP también podría utilizarse si la aplicación necesitara consultar una API o enviar datos a un servicio web; la elección depende de los requisitos, la infraestructura y la seguridad requerida.

9. QoS recomendado por tópico

Los niveles QoS controlan la garantía de entrega de mensajes MQTT. La evidencia aportada no muestra el QoS configurado en el código, por lo que la siguiente tabla es una recomendación de diseño, no una afirmación sobre la configuración actual.

Tópico / dato	QoS sugerido	Justificación
Temperatura	0	Medición frecuente; una pérdida puede compensarse con la siguiente lectura.
Humedad	0	Dato periódico no crítico para la demostración.
Luminosidad	0	Las actualizaciones posteriores reemplazan la lectura anterior.
Actuadores / LED o buzzer	1	Busca entregar el comando al menos una vez; el receptor debe tolerar duplicados.
Estado	1	Es útil reducir la posibilidad de perder cambios de estado.

QoS 0 entrega como máximo una vez y no confirma la recepción; QoS 1 entrega al menos una vez y puede generar duplicados; QoS 2 proporciona una entrega exactamente una vez a nivel del protocolo, con mayor intercambio de mensajes. Para los comandos de actuadores, también es importante que el programa evite ejecutar dos veces una acción no idempotente.

10. Flujo de datos del sistema

Flujo implementado y evidenciado: sensor(es) → NodeMCU ESP8266 → Wi-Fi → broker MQTT (broker.hivemq.com) → MQTT Explorer / clientes suscritos.

Ampliación propuesta: broker MQTT → servicio de integración (por ejemplo, Node-RED o una plataforma compatible) → dashboard; y broker MQTT → servicio suscriptor → base de datos con fecha y hora → consulta histórica y gráficos. Estos componentes adicionales no quedan demostrados en las imágenes entregadas.

Para controlar un actuador, el flujo sería inverso: aplicación o cliente → broker MQTT → tópico del actuador → microcontrolador → LED o buzzer. El dispositivo podría publicar después su estado para confirmar el cambio.

11. Procedimiento de prueba y resultados

Procedimiento documentado a partir de las evidencias:

1. Se ejecutó el programa en Arduino IDE con una placa NodeMCU 1.0 (ESP-12E).
2. El monitor serial mostró lecturas de temperatura, humedad y luminosidad.
3. Se observaron mensajes recibidos en tópicos de otros equipos.
4. MQTT Explorer mostró el broker HiveMQ y los tópicos del equipo, con valores de estado y sensores.
5. Las capturas permiten comprobar intercambio de mensajes durante la ejecución.

Resultados observados: temperatura alrededor de 29,4–29,9 °C en las capturas; humedad de aproximadamente 59 %; luminosidad igual a 0; y estado del dispositivo reportado como online en MQTT Explorer. Los valores son los visibles en las imágenes y pueden variar en otras ejecuciones.

Limitaciones: las imágenes no permiten confirmar todos los pines, la referencia exacta de cada sensor, la frecuencia de publicación, el QoS realmente configurado ni la existencia de base de datos o dashboard. Esos aspectos deben verificarse en el código y en pruebas adicionales.

12. Preguntas de reflexión

12.1. ¿Por qué MQTT puede ser más eficiente que HTTP en IoT?
Porque utiliza mensajes pequeños y un modelo publicador/suscriptor que facilita distribuir datos a varios clientes. Esto resulta útil para mediciones periódicas y dispositivos con recursos limitados. HTTP sigue siendo adecuado para APIs y solicitudes web.

12.2. ¿Qué ocurre si el broker deja de funcionar?
Los clientes dejan de intercambiar mensajes a través de ese broker. El microcontrolador debe detectar la desconexión e intentar reconectarse. Los datos generados durante la caída pueden perderse si no se almacenan localmente.

12.3. ¿Qué QoS conviene para temperatura y actuadores?
QoS 0 suele bastar para lecturas periódicas de temperatura; QoS 1 es recomendable para comandos de actuadores cuando interesa que lleguen al menos una vez, controlando posibles duplicados.

12.4. ¿Qué es MQTTS?
Es MQTT protegido mediante TLS, que cifra la comunicación y ayuda a verificar la identidad del servidor. Debe configurarse con los puertos y certificados admitidos por el broker.

12.5. ¿Cómo se relaciona con un proyecto integrador?
Integra electrónica, programación y redes: los sensores generan datos, el microcontrolador los procesa y MQTT los distribuye a aplicaciones. Un dashboard y un histórico ampliarían el análisis de las mediciones.

13. Conclusiones

La evidencia presentada respalda el funcionamiento de una práctica IoT en la que una placa NodeMCU basada en ESP8266 publica datos ambientales mediante MQTT y permite observarlos en MQTT Explorer. El monitor serial también muestra mensajes recibidos de otros equipos, por lo que se documentan tanto la publicación como la recepción de datos.

MQTT resulta adecuado para este escenario por su modelo publicador/suscriptor y su eficiencia en mensajes periódicos. Para completar la documentación con mayor precisión, es importante mantener la identificación correcta de la placa, confirmar los sensores y pines, y distinguir los elementos efectivamente probados de las ampliaciones propuestas. La integración de un dashboard o una base de datos sería un paso posterior, no demostrado por las capturas actuales.

14. Herramientas utilizadas

- Arduino IDE 2.3.10, para editar y ejecutar el programa.
- Placa NodeMCU 1.0 basada en ESP8266.
- Biblioteca PubSubClient, visible en el entorno de desarrollo.
- Monitor serial del Arduino IDE, para revisar lecturas y mensajes.
- MQTT Explorer, para visualizar tópicos y payloads.
- Broker público broker.hivemq.com, para intercambio de mensajes MQTT.

15. Referencias

Arduino. (s. f.). *Arduino IDE*. https://www.arduino.cc/en/software

HiveMQ. (s. f.). *MQTT Essentials*. https://www.hivemq.com/mqtt-essentials/

OASIS. (2019). *MQTT Version 5.0*. https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

Espressif Systems. (s. f.). *ESP8266EX Datasheet*. https://www.espressif.com/en/support/documents/technical-documents

O'Leary, N. (s. f.). *PubSubClient: MQTT client library for Arduino* [Repositorio de software]. GitHub. https://github.com/knolleary/pubsubclient

Nota final: el informe incorpora las dos evidencias proporcionadas. Los valores descritos proceden de lo visible en las capturas; las configuraciones no visibles se señalan como recomendaciones para evitar afirmar resultados que no se han demostrado.